A PROJECT REPORT

on

CLOUD WORKLOAD PREDICTION USING MACHINE LEARNING

Submitted in partial fulfillment of the requirements for the award of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

Submitted by

NEMALA LAKSHMI POOJITHA – 22BQ1A12A7

KANCHARLA SAI SAHITHI – 22BQ1A1265

MADDU ARUNA – 22BQ1A1290

PALEPU LAKSHMI NAVYA – 22BQ1A12B4

Under the esteemed guidance of

Dr. S. Mallikarjun Rao M. Tech,Ph.D;

Associate Professor

DEPARTMENT OF INFORMATION TECHNOLOGY
VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY
NAMBUR (V), PEDAKAKANI (M), GUNTUR-522 508, TEL no: 0873 2118036,

www.vvitguntur.com, approved by AICTE, permanently affiliated to JNTUK
Accredited by NAAC with "A" grade, Accredited by NBA for 3 years

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY:: NAMBUR

BONAFIDE CERTIFICATE

Certified that this project report "**CLOUD WORKLOAD PREDICTION USING MACHINE LEARNING**" is the bonafide work of by "**NEMALA LAKSHMI POOJITHA (22BQ1A12A7), KANCHARLA SAI SAHITHI (22BQ1A1265), MADDU ARUNA(22BQ1A1290), PALEPU LAKSHMI NAVYA (22BQ1A12B4)**" who carried out the project under my guidance during the year 2025-26 towards partial fulfillment of the requirements of the Degree of Bachelor of Technology in Information Technology from Jawaharlal Nehru Technological University, Kakinada. The results embodied in this report have not been submitted to any other University for the award of any degree.

Signature of the Supervisor

Dr S.MALLIKARJUN RAO
Associate Professor, IT.

Signature of the Head of the Department

Dr KALAVATHI ALLA
Professor, IT.

Submitted for Viva voce Examination held on _____

EXTERNAL EXAMINER

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY:: NAMBUR

CERTIFICATE OF AUTHENTICATION

We solemnly declare that this project report **"CLOUD WORKLOAD PREDICTION USING MACHINE LEARNING"** is the bonafide work done purely by us, carried out under the supervision of Dr. S. MALLIKARJUN RAO, towards partial fulfillment of the requirements of the Degree of Bachelor of Technology in Information Technology from Jawaharlal Nehru Technological University, Kakinada during the year 2025-26.

It is further certified that this work has not been submitted, either in part of in full, to any other department of the Jawaharlal Nehru Technological University, or any other University, institution or elsewhere, or for publication in any form.

Signature of the Students

NEMALA LAKSHMI POOJITHA – 22BQ1A12A7_____

KANCHARLA SAI SAHITHI – 22BQ1A1265_____

MADDU ARUNA – 22BQ1A1290_____

PALEPU LAKSHMI NAVYA – 22BQ1A12B4_____

ACKNOWLEDGEMENT

We take this opportunity to express our deepest gratitude and appreciation to all those people who made this project work easier with words of encouragement, motivation, discipline, and faith by offering different places to look to expand my ideas and help me towards the successful completion of this project work.

First and foremost, we express our deep gratitude to **Mr. Vasireddy Vidyasagar**, Chairman, Vasireddy Venkatadri Institute of Technology for providing necessary facilities throughout the Information Technology program.

We express our sincere thanks to **Dr. Y. Mallikarjuna Reddy**, Principal, Vasireddy Venkatadri Institute of Technology for his constant support and cooperation throughout the Information Technology program.

We express our sincere gratitude to **Dr. A. Kalavathi**, Professor & HOD, Information Technology, Vasireddy Venkatadri Institute of Technology for her constant encouragement, motivation and faith by offering different places to look to expand my ideas. We would like to express our sincere gratitude to our guide **Dr. S. Mallikarjun Rao** for his insightful advice, motivating suggestions, invaluable guidance, help and support in successful completion of this project.

We would like to take this opportunity to express our thanks to the **teaching and non- teaching** staff in the Department of Information Technology, VVIT for their invaluable help and support.

NEMALA LAKSHMI POOJITHA
KANCHARLA SAI SAHITHI
MADDU ARUNA
PALEPU LAKSHMI NAVYA

ABSTRACT

In modern cloud computing environments, resource usage patterns such as CPU load, memory consumption, disk I/O, and network throughput often exhibit highly dynamic behaviour. These fluctuations become even more pronounced in large-scale organizations where hundreds or even thousands of users may simultaneously request virtual machines, execute computational tasks, or access cloud-hosted applications. As a result, cloud infrastructure must continuously adapt to varying demand levels, and failure to do so can introduce serious challenges.

One critical challenge is over-utilization, where the workload exceeds the available resource capacity. This may lead to degraded application performance, service interruptions, increased latency, and even system crashes—directly affecting user experience and violating Service Level Agreements (SLAs). On the other hand, under-utilization is equally problematic, as it results in idle or underused resources, driving up unnecessary operational costs for cloud providers and organizations. Maintaining the balance between these two extremes is essential for achieving cost-effective and efficient cloud operations.

To address these challenges, this project proposes a Cloud Workload Prediction System that leverages machine learning algorithms, specifically Decision Trees and Random Forests, to forecast future workload conditions. The system uses historical time-series workload data, incorporating key performance indicators such as CPU usage, memory consumption, disk read/write cycles, and network traffic rates. These features capture both short-term spikes and long-term behavioural patterns within the cloud environment.

The machine learning models are trained to classify whether the cloud workload is likely to experience over-utilization or under-utilization within the next 24 hours. By converting continuous workload metrics into predictive utilization categories, the system enables administrators and cloud orchestration platforms to take proactive measures. These include dynamic resource allocation, automatic scaling up or scaling down of virtual machines, and intelligent distribution of load across servers.

TABLE OF CONTENTS

1 INTRODUCTION

1.1 Introduction. **01**

1.2 Problem Statement. **02**

2 SYSTEM ANALYSIS

2.1 Requirement Analysis **03**

2.1.1 Nonfunctional requirements. **03**

2.2 Hardware requirements. **04**

2.3 Software requirements. **05**

2.4 Existing System. **06**

2.5 Proposed System. **07**

2.6 Modules. **07**

3 SYSTEM DESIGN

3.1 System Architecture. **09**

3.2 UML Diagrams. **10**

3.2.1 Use case diagram. **11**

3.2.2 Sequence diagram. **13**

3.2.3 Class diagram. **14**

3.2.4 Activity diagram. **16**

4 SYSTEM IMPLEMENTATION

4.1 Technologies Used. **17**

4.1.1 Python Based Machine Learning Stack **17**

4.1.2 Frontend in web development. **19**

4.1.3 Back-end using Java Servlets. **19**

4.1.4 Development Environment. **21**

4.2 Approach & Implementation 22
4.2.1 Backend Logic. 22
4.2.1.1 preprocess.py. 30
4.2.1.2 train_model.py. 31
4.2.1.3 metrics.py. 31
4.2.2 Frontend Structure. 33
4.2.2.1 app.py. 33
4.2.2.2 Index.html. 35
4.2.3 Deployment. 37
4.3 Outcome of Project. 38

5 SYSTEM TESTING

5.1 Purpose of Testing. 43
5.2 Testing Strategies. 43
5.2.1 Unit Testing. 44
5.2.2 Integration Testing. 45
5.2.3 System Testing. 45

6 CONCLUSION

6.1 Conclusion. 47
6.2 Future Scope. 48

CHAPTER 1 - INTRODUCTION

1.1 INTRODUCTION

Cloud computing has emerged as the foundational technology for modern organizations, enabling them to access and utilize computing resources—such as CPU power, memory, storage, and network bandwidth—on demand. Unlike traditional on-premise infrastructures, cloud environments are designed to be elastic, scalable, and cost-efficient, making them suitable for handling diverse workloads across business, research, and technological domains. However, despite these advantages, resource utilization in cloud systems is inherently unpredictable. Workload patterns fluctuate rapidly throughout the day due to variations in user activity, application behaviour, scheduled tasks, and unexpected traffic spikes.

These dynamic variations introduce two major operational challenges. The first is **over-utilization**, where resource demand exceeds available capacity. This can lead to degraded system performance, increased response time, service interruptions, and, ultimately, user dissatisfaction. The second challenge is **under-utilization**, where resources remain idle or underused, causing unnecessary financial expenditure and inefficient energy consumption. In large-scale cloud environments, managing these fluctuations manually becomes impractical and inefficient.

To mitigate these challenges, cloud service providers increasingly rely on predictive analytics and intelligent automation. This project contributes to that advancement by developing a **cloud workload prediction system using machine learning algorithms**, specifically Decision Trees and Random Forests. These models are trained on historical cloud workload datasets containing key performance metrics such as CPU usage, memory consumption, disk I/O operations, and network activity. By learning from past utilization patterns, the system aims to accurately forecast the workload behaviour for the next 24 hours.

Overall, this project demonstrates how machine learning can enhance the adaptability, resilience, and cost-effectiveness of cloud environments. By integrating prediction models into resource management workflows, cloud providers can ensure more consistent service quality while significantly improving the utilization of their infrastructure.

1.1 PROBLEM STATEMENT

Cloud environments operate under highly dynamic and unpredictable conditions where key resources—CPU, memory, disk I/O, and network bandwidth—fluctuate continuously with user demand. These variations create major operational challenges. When resources become over-utilized, the system may experience performance degradation, higher response times, and even service outages, ultimately affecting user satisfaction and violating Service Level Agreements.

Conversely, **under-utilization** of resources leads to unnecessary operational costs, as organizations continue paying for idle or underused virtual machines. This inefficiency becomes more significant in large-scale cloud infrastructures with thousands of running instances.

Most existing cloud systems rely on **static threshold-based resource allocation**, such as scaling out when CPU usage crosses a fixed limit. These techniques fail to adapt to rapidly changing workloads influenced by time-of-day trends, user behavior, or unexpected traffic spikes. As a result, organizations either over-provision resources or fail to scale quickly enough to meet demand.

Balancing cost efficiency with reliable performance remains a major challenge, especially when workload patterns are unpredictable. A key limitation is the **lack of accurate workload prediction**, which keeps auto-scaling mechanisms reactive rather than proactive.

Thus, there is a clear need for an intelligent prediction system that analyzes historical workload data and forecasts future resource demands. Accurate workload prediction enables proactive auto-scaling, improves performance, reduces wastage, and enhances the overall cost-efficiency of cloud environments.

CHAPTER II
SYSTEM ANALYSIS

2.1 REQUIREMENT ANALYSIS

2.1.1 Non-Functional Requirements

Performance Requirements :

The system must process historical workload logs and generate predictions with low latency. The prediction time for a single input instance (e.g., hourly CPU/memory data) should be minimal to ensure real-time or near–real-time decision-making. The machine learning model should efficiently handle large datasets containing several months or years of cloud resource utilization logs.

Software Quality Attributes : Accuracy

The prediction accuracy of the machine learning models (Decision Trees and Random Forests) must surpass that of traditional threshold-based approaches. The classification of workload states (over-utilized, under-utilized, or normal) should achieve high values in metrics such as Accuracy, Precision, Recall, and F1-score to ensure reliable forecasting.

Reliability :

The system must consistently deliver accurate workload predictions without failure during execution. It should be resilient to noisy or incomplete logs by implementing preprocessing techniques that ensure stable model outputs. The prediction module must not produce incorrect or corrupted results during high-load analysis.

Scalability :
The application architecture must be capable of handling growing volumes of cloud workload data as organizations expand their infrastructure.The system should support increasing numbers of workload metrics (CPU, memory, disk I/O, network I/O) without degrading performance. Scalability must also ensure that the model can be retrained periodically as new data will available.

2.2 HARDWARE REQUIREMENTS

For optimal performance in both training and inference, the project requires specific hardware resources, particularly a GPU for accelerating deep learning computations.

Minimum Hardware Specifications

- Network: 50 Mbps or higher
- Storage: 50 GB or more (for workload logs, model files, evaluation reports)
- RAM: 8 GB or higher
- Processor: Intel Core i5 or above
- GPU (Optional but beneficial): NVIDIA GTX series or higher for faster model training

Recommended Hardware Specifications (For Large Datasets)

- Network: 100 Mbps for continuous data ingestion
- Storage: 100 GB or higher
- RAM: 16 GB or more for handling high-dimensional workload logs
- Processor: Intel Core i7 or above
- GPU: NVIDIA RTX series (e.g., RTX 3060 / 4060 / 4070) for accelerated training of Random Forests on large datasets

2.3 SOFTWARE REQUIREMENTS

The project requires a software environment capable of handling machine learning model training, cloud workload forecasting, and web-based deployment for seamless user interaction.

Backend Requirements (FastAPI)

- **Python (3.8 or higher):**
  The primary programming language for building the workload prediction system.

- **Machine Learning Libraries:**
  Scikit-learn: Used for training Random Forest models.

  NumPy & Pandas: For dataset preprocessing, feature engineering.

  Matplotlib / Seaborn: For generating plots during model evaluation.

- **Backend Framework – FastAPI:**
  FastAPI is used to build fast, asynchronous REST APIs that handle:

  - Model inference requests
  - Workload prediction endpoints
  - Communication between the frontend and ML models
  - Deployment of trained prediction models

- **Joblib / Pickle:**
  Used for storing and loading trained machine learning models.

Frontend Requirements (React.js)

- **React.js:**

Provides a dynamic and responsive user interface for interacting with prediction system.

- **UI Libraries:**

  Material UI / Bootstrap: For designing professional, responsive dashboards.

  Axios: For making API calls to the FastAPI backend.

- **Charts for Visualization:**

  Chart.js / Recharts: Used to visualize workload patterns, daily trends, and predicted values.

Database Requirements

- **CSV Files:**
  Used for storing workload logs, training datasets, and prediction results.

- **JSON Files:**
  Used for configuration settings or model metadata if needed.

- **Local / Cloud Storage:**
  To store processed workload logs, model files, and system outputs.
  (*Optional:* Can be extended to store data in MongoDB, PostgreSQL, or AWS S3 if the project is scaled.)

Development Environment

- **IDE (Integrated Development Environment): VS Code**
  Used for coding, debugging, and testing both backend and frontend components.
- **Version Control System: Git / GitHub**
  Manages code versions, collaboration, and deployment tracking.
- Python Libraries like NumPy & Pandas, Scikit-learn, Matplotlib/ Seaborn, FastAPI and uvicorn .

2.4 EXISTING SYSTEM

The existing landscape for cloud workload prediction primarily relies on traditional resource allocation techniques such as static threshold-based rules and manual scaling policies. In these systems, cloud services scale resources only after certain fixed conditions are met—for example, scaling out when CPU usage exceeds 70% or scaling in when usage drops below 30%. While simple to implement, these reactive methods struggle to handle highly dynamic and rapidly fluctuating workloads.

Several earlier approaches have attempted to improve prediction accuracy using conventional methods such as **linear regression, moving averages, ARIMA models, and heuristic-based algorithms**. Although these approaches provide basic trend analysis, they fail to capture non-linear workload patterns, sudden spikes, or complex relationships among metrics like CPU, memory, network I/O, and disk activity.

More recent research has introduced machine learning techniques—such as Support Vector Machines (SVMs), Naïve Bayes, and basic Decision Trees—for workload prediction. These models achieve moderate accuracy but still face challenges in handling large-scale datasets, multi-dimensional cloud metrics, and the high variability of real-time workloads. Additionally, many existing methods lack interpretability and are not optimized for proactive auto-scaling decisions.

Because of these limitations, current systems often struggle with **resource over-provisioning**, **under-utilization**, and inconsistent performance. This creates a strong need for more robust, accurate, and adaptive approaches. Machine learning models like **Decision Trees and Random Forests**, which are designed to capture non-linear interactions and complex workload behaviors, offer a more reliable solution for predicting future cloud resource utilization and enabling cost-effective, proactive scaling.

2.4 PROPOSED SYSTEM

The proposed **Cloud Workload Prediction Using Machine Learning** system addresses the limitations of existing static and reactive resource allocation approaches by introducing a predictive, intelligent, and data-driven solution. Unlike traditional methods that rely on fixed thresholds or manual scaling, the proposed system uses machine learning algorithms—specifically **Decision Trees and Random Forests**—to analyze historical cloud workload data and forecast future resource utilization.

2.5 MODULES

1. Data Acquisition / Ingestion Module

This module collects historical cloud workload logs such as CPU usage, memory consumption, disk I/O and network metrics. It loads CSV/JSON traces or retrieves data from cloud monitoring APIs like AWS CloudWatch or Google Stackdriver. The module stores the ingested data in Pandas DataFrames for downstream processing. Key tasks include file input/output operations, timestamp parsing, validating data integrity, and indexing metrics by time for accurate time-series analysis.

2. Exploratory Data Analysis (EDA) Module

This module provides statistical and visual summaries of the workload time-series data. It generates plots showing trends in CPU and memory usage, histograms of distributions, autocorrelation graphs, and seasonal pattern visualizations. Missing-value detection and anomaly identification are also performed. EDA helps in understanding the behavior of workloads over time and guides the design of relevant features that improve model accuracy.

3. Data Preprocessing & Feature Engineering Module

This module cleans the raw workload logs by imputing or removing missing values, eliminating duplicates, and normalizing numerical features. It also constructs essential time-series features such as lag variables (t–1, t–24), rolling averages, rate-of-change indicators, and time-based features like hour-of-day or day-of-week. These engineered features enhance model stability and performance by enabling the prediction model to capture temporal patterns effectively.

4. Train / Validation Split & Time-Series Cross-Validation Module

This module divides the dataset into training, validation, and testing sets using time-aware splitting to avoid look-ahead bias. Unlike random shuffling, the data is split chronologically. Rolling or expanding window cross-validation techniques are used to evaluate model generalization on future unseen data. This ensures that the model is trained and validated in a realistic time-series forecasting environment.

5. Model Definition & Training Module

This module defines, trains, and saves machine learning models such as Decision Trees and Random Forests. It includes configuration of hyperparameters, encoding workload classes (Over-Utilized, Under-Utilized, Normal), handling class imbalance, and using Scikit-learn functions to fit the models. Optional grid search or randomized search may be used to find optimal model parameters for improved accuracy.

6. Model Evaluation & Metrics Module

This module evaluates the trained models using appropriate performance metrics. For classification, it computes accuracy, precision, recall, F1-score, and confusion matrices. If continuous utilization prediction is used, regression metrics like MAE and RMSE are calculated. It also produces overlay plots of actual vs. predicted values and ROC/PR curves to visually assess model performance across different conditions.

7. Prediction / Inference Module

This module loads previously trained models (saved in joblib/pickle format) and applies the same preprocessing pipeline to new incoming workload data. It generates predictions for the next time interval (e.g., next hour or next 24 hours). This is the core module used by APIs or automated systems that need real-time forecasts for immediate decision-making.

8. Decision Engine / Auto-Scaling Rule Module

This module converts model predictions into actionable scaling decisions. For example, if the predicted probability of over-utilization exceeds a threshold, the system recommends scaling out. If under-utilization is expected, it recommends scaling in to reduce wastage. The module may include safety checks, cooldown periods, and integration with cloud provider APIs to automate scaling operations.

9. Visualization & Reporting Module

This module generates visual reports and dashboards that present time-series plots of workload metrics, prediction results, and classification distributions. Tools such as Matplotlib, Seaborn, or Plotly are used. It also provides exportable reports in CSV or JSON formats, helping stakeholders understand workload patterns and evaluateoko model predictions easily.

10. Model Persistence & Experiment Tracking Module

This module is responsible for saving trained models, preprocessing transformers, and metadata (such as hyperparameters and accuracy scores). It uses joblib/pickle for model persistence and may integrate with MLflow or similar tools for experiment tracking, allowing developers to compare multiple training runs and select the best model.

CHAPTER III
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The system architecture for the Cloud Workload Prediction Using Machine Learning project represents a structured and modular design that defines how different components of the workload prediction system interact with each other. It follows a multi-layered approach to ensure scalability, reliability, and efficient processing of cloud resource data. The architecture supports the complete workflow—from data collection to prediction-based decision-making. The flow of data progresses sequentially through key stages: ingestion of historical workload logs, preprocessing and feature engineering, machine learning model training, prediction and inference, and finally, decision generation for proactive auto-scaling. Each module is designed to operate independently yet integrates smoothly, enabling the system to accurately forecast cloud resource utilization and support optimized scaling actions.

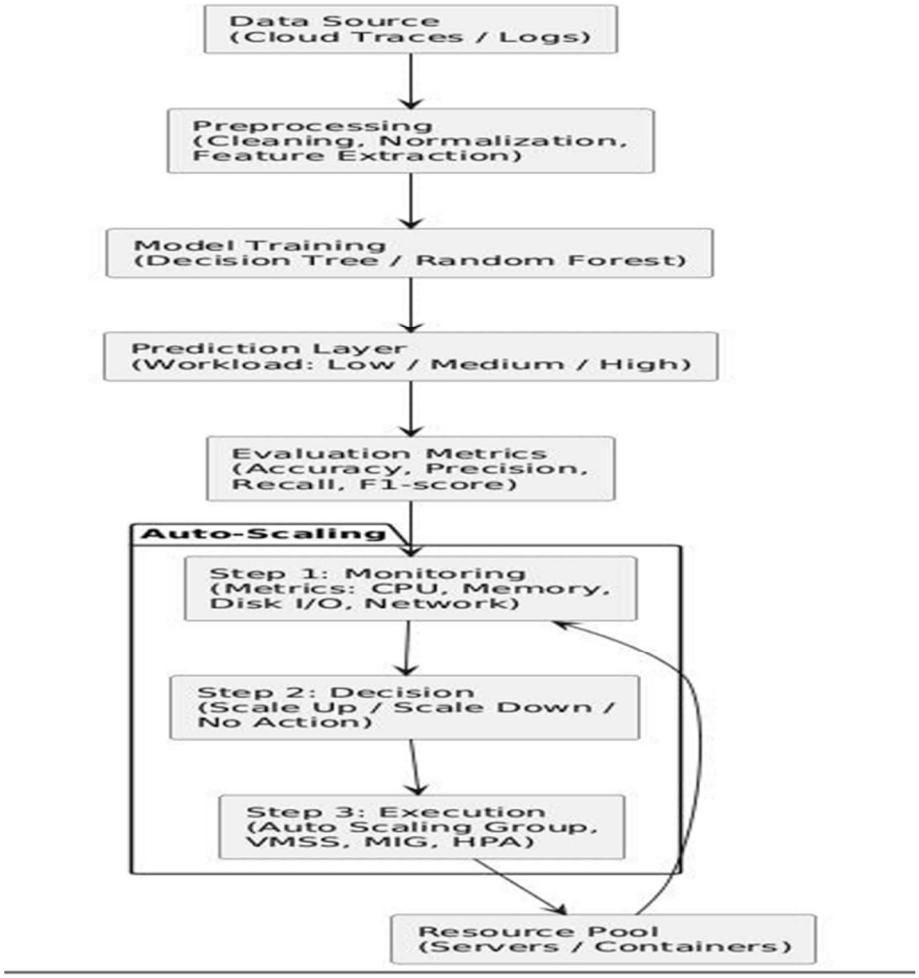


Fig 3.1 System Architecture

3.1 UML DIAGRAMS

UML is an acronym that stands for **Unified Modeling Language**. Simply put, UML is a modern approach to modeling and documenting software. In fact, it's one of the most popular business process modeling techniques.

It is based on diagrammatic representations of software components. As the old proverb says: "a picture is worth a thousand words". By using visual representations, we are able to better understand possible flaws or errors in software or business processes.

Mainly, UML has been used as a general-purpose modeling language in the field of software engineering. However, it has now found its way into the documentation of several business processes or workflows. For example, activity diagrams, a type of UML diagram, can be used as a replacement for flowcharts. They provide both a more standardized way of modeling workflows as well as a wider range of features to improve readability and efficacy.

Any complex system is best understood by making some kind of diagrams or pictures. These diagrams have a better impact on our understanding. There are two broad categories of diagrams and they are again divided into subcategories –

- Structural Diagrams
- Behavioral Diagrams

Structural Diagrams

The structural diagrams represent the static aspect of the system. These static aspects represent those parts of a diagram, which forms the main structure and are therefore stable.

These static parts are represented by classes, interfaces, objects, components, and nodes. The four structural diagrams are −

- Class diagram
    - Object diagram
    - Component diagram
    - Deployment diagram

Behavioral Diagrams

Any system can have two aspects, static and dynamic. So, a model is considered as complete when both the aspects are fully covered. Behavioral diagrams basically capture the dynamic aspect of a system. UML has the following five types of behavioral diagrams

- Use case diagram
- Sequence diagram
- Collaboration diagram
- Statechart diagram
- Activity diagram

3.2.1 USE CASE DIAGRAM

A use case diagram at the simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved. A use case diagram can identify the different types of users of a system and the different use cases and will often be accompanied by other types of diagrams as well. The use cases are represented by either circles or ellipses.

While a use case itself might drill into a lot of detail about every possibility, a use-case diagram can help provide a high-level view of the system. It has been said before that "Use case diagrams are the blueprints for your system". They provide a simplified and graphical representation of what the system must actually do.

Due to their simplistic nature, use case diagrams can be a good communication tool for stakeholders. The drawings attempt to mimic the real world and provide a view for the stakeholder to understand how the system is going to be designed.

The purpose of the use case diagram is simply to provide the high-level view of the system and convey the requirements in laypeople's terms for the stakeholders. Additional diagrams and documentation can be used to provide a complete functional and technical view of the system.
Use case diagrams are used to gather the requirements of a system including internal and external

influences. These requirements are mostly design requirements. Hence, when a system is analyzed to gather its functionalities, use cases are prepared and actors are identified.

When the initial task is complete, use case diagrams are modelled to present the outside view. In brief, the purpose of use case diagrams can be said to be as follows –

- Used to gather the requirements of a system.
- Used to get an outside view of a system.
- Identify the external and internal factors influencing the system.
- Shows the interactions among the requirements are actors.

3.1.1 Use-case diagram:

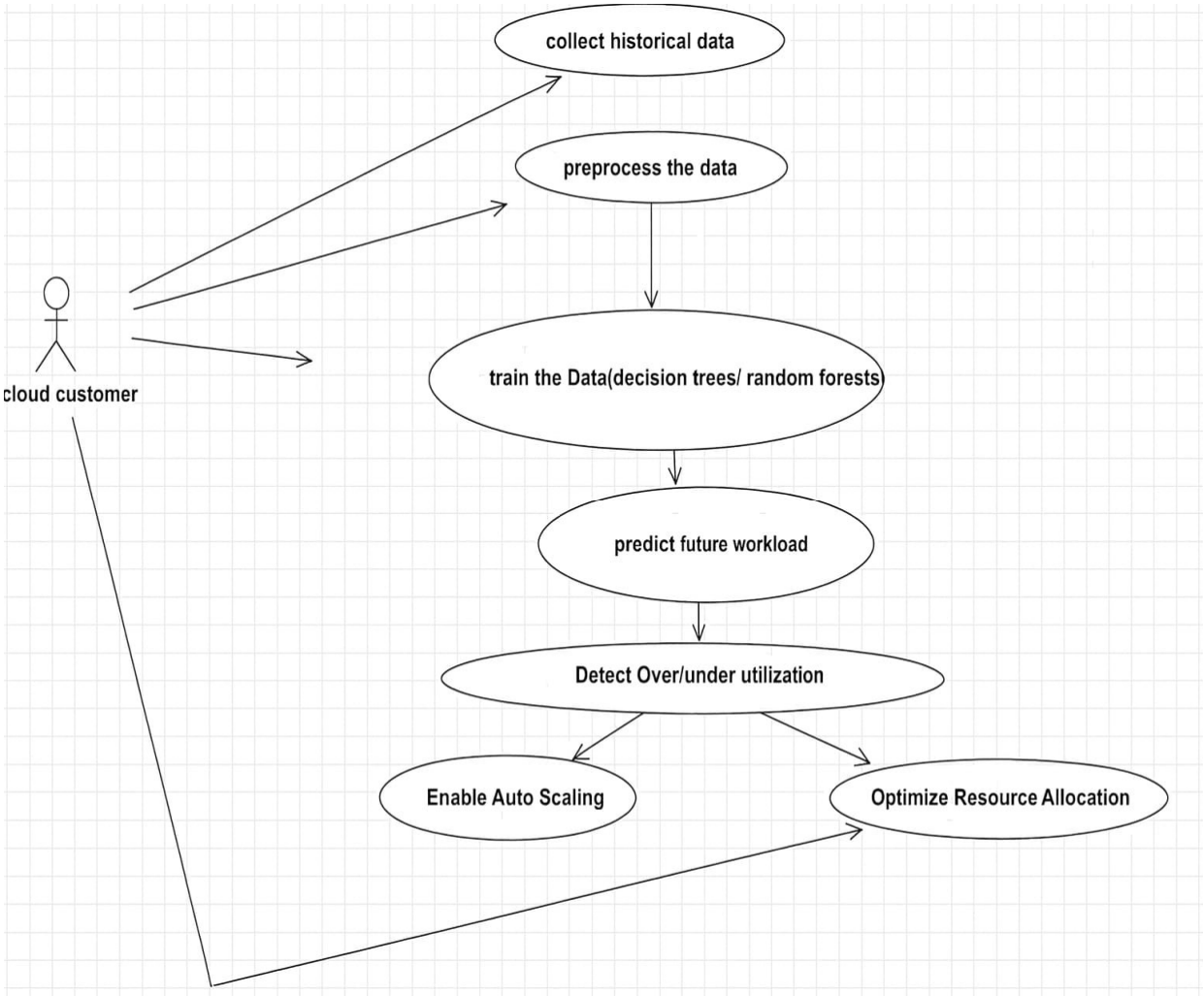


Fig 3.2.1 Use-case diagram

3.1.2 SEQUENCE DIAGRAM

A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the Logical View of the system under development. Sequence diagrams are sometimes called event diagrams or event scenarios.

A sequence diagram shows, as parallel vertical lines (*lifelines*), different processes or objects that live simultaneously, and, as horizontal arrows, the messages exchanged between them, in the order in which they occur. This allows the specification of simple runtime scenarios in a graphical manner.

If the lifeline is that of an object, it demonstrates a role. Leaving the instance name blank can represent anonymous and unnamed instances.

Messages, written with horizontal arrows with the message name written above them, display interaction. Solid arrowheads represent synchronous calls, open arrowheads represent asynchronous messages, and dashed lines represent reply messages. If a caller sends a synchronous message, it must wait until the message is done, such as invoking a subroutine. If a caller sends an asynchronous message, it can continue processing and doesn't have to wait for a response.

Asynchronous calls are present in multithreaded applications, event-driven applications and in message-oriented middleware. Activation boxes, or method-call boxes, are opaque rectangles drawn on top of lifelines to represent that processes are being performed in response to the message (Execution Specifications in UML).

Objects calling methods on themselves use messages and add new activation boxes on top of any others to indicate a further level of processing. If an object is destroyed (removed from memory), an X is drawn on the bottom of the lifeline, and the dashed line ceases to be drawn below it.

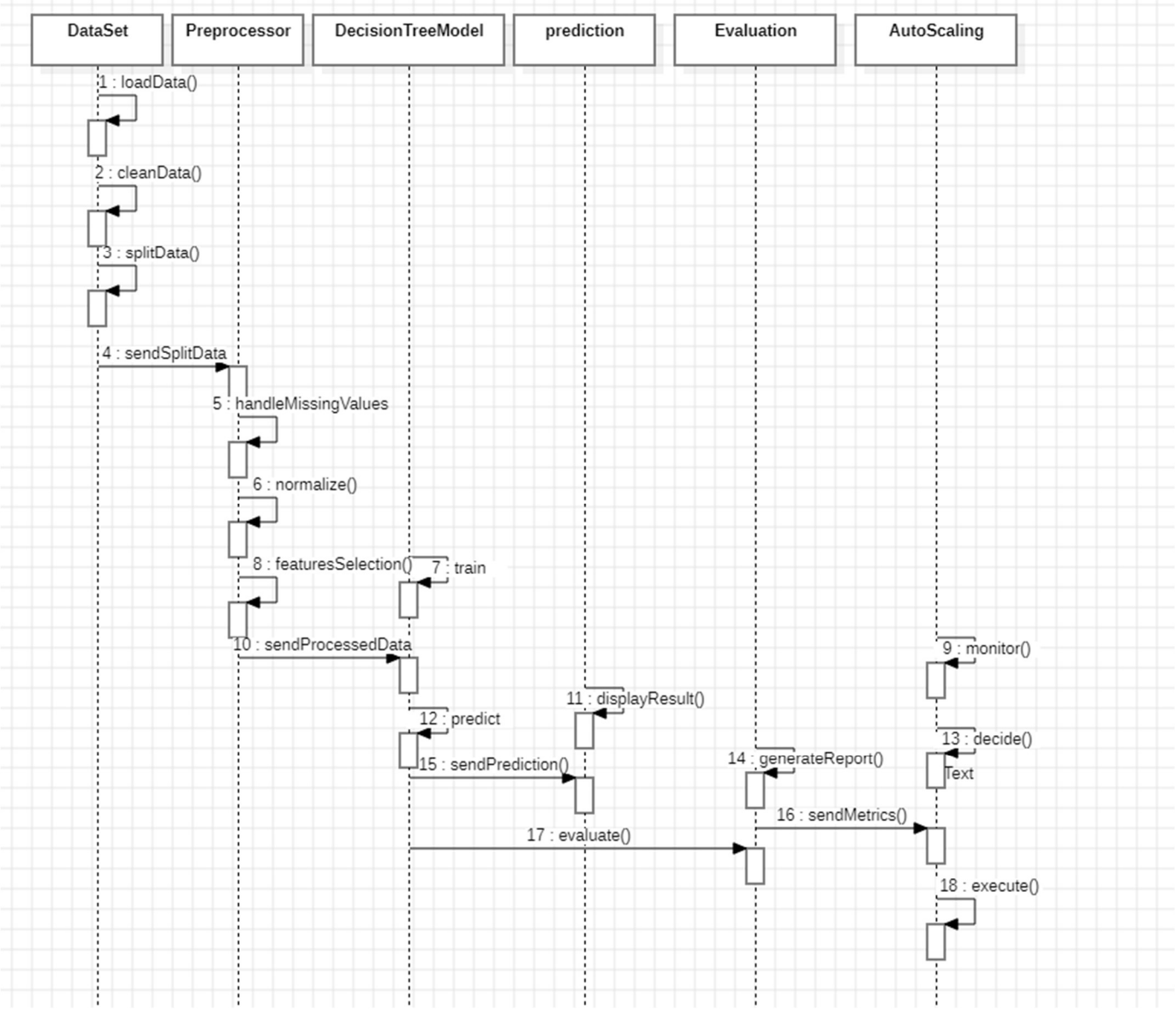


Fig 3.2.2 Sequence Diagram

3.1.3 CLASS DIAGRAM

A class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects.

The Class Diagram is the main building block of object- oriented modeling. It is used for general conceptual modeling of the structure of the application, and for detailed modeling translating the models into programming code. Class diagrams can also be used for data modeling. The classes in a class diagram represent both the main elements, interactions in the application, and the classes to be programmed.

In the diagram, classes are represented with boxes that contain three compartments:

- The top compartment contains the name of the class. It is printed in bold and centered, and the first letter is capitalized.
- The middle compartment contains the attributes of the class. They are left-aligned and the first letter is lowercase.
- The bottom compartment contains the operations the class can execute. They are also left-aligned and the first letter is lowercase.

In this project, we are considering the user and system as classes. Each of them has their own set of attributes and relevant operations as shown.

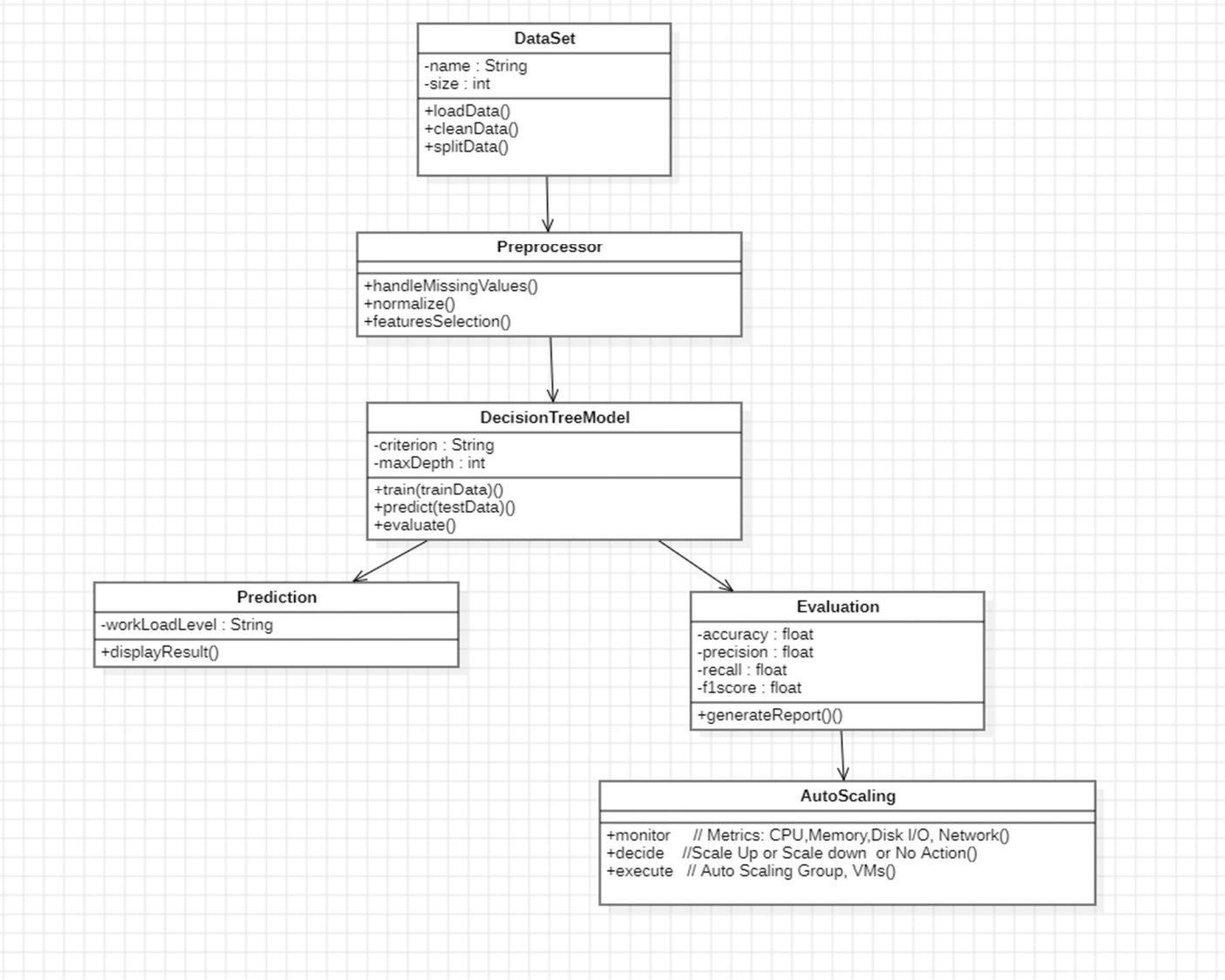


Fig 3.2.3 Class Diagram

3.1.4 ACTIVITY DIAGRAM

We use **Activity Diagrams** to illustrate the flow of control in a system and refer to the steps involved in the execution of a use case. We model sequential and concurrent activities using activity diagrams. So, we basically depict workflows visually using an activity diagram. An activity diagram focuses on condition of flow and the sequence in which it happens. We describe or depict what causes a particular event using an activity diagram. UML models basically three types of diagrams, namely, structure diagrams, interaction diagrams, and behavior diagrams. An activity diagram is a **behavioral diagram** i.e. it depicts the behavior of a system. An activity diagram portrays the control flow from a start point to a finish point showing the various decision paths that exist while the activity is being executed. We can depict both sequential processing and concurrent processing of activities using an activity diagram. They are used in business and process modelling where their primary use is to depict the dynamic aspects of a system. An activity diagram is very **similar to a flowchart**

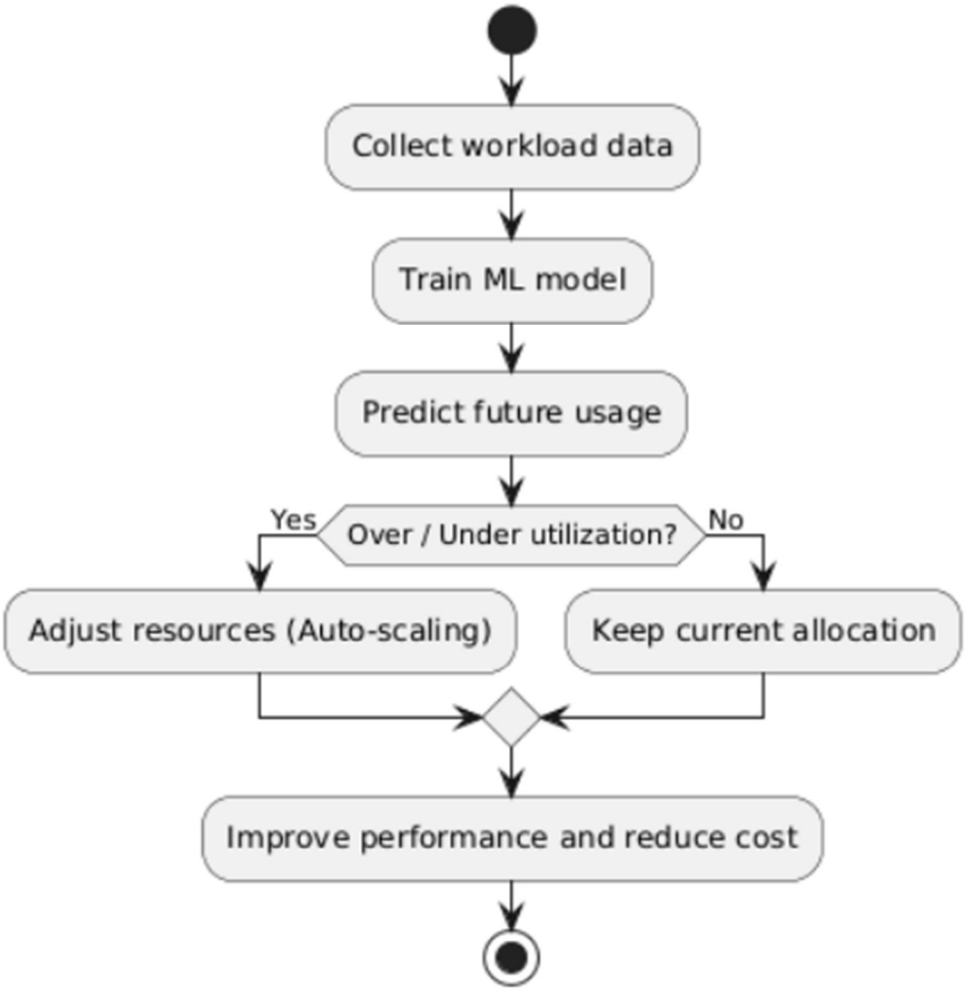


Fig 3.2.4 Activity Diagram

CHAPTER IV

SYSTEM IMPLEMENTATION

4.1 TECHNOLOGIES USED:

4.1.1 Python-Based Machine Learning Stack

Python serves as the core platform for implementing and deploying the Cloud Workload Prediction System. The technology stack integrates powerful machine learning, data processing, and visualization libraries to ensure high prediction accuracy and efficient execution. The main components include:

• **Machine Learning Framework (Scikit-Learn)**

Scikit-Learn is used to build and train the core machine learning models such as Decision Trees and Random Forests. It enables- Easy experimentation with hyperparameters, Handling multi-class classification (Over-utilized, Under-utilized, Normal), Efficient model training and evaluation, Preprocessing pipeline support.

Random Forests offer robustness and effectively capture non-linear workload patterns, making them well-suited for cloud workload forecasting.

• **Data Processing (Pandas and NumPy)**

These libraries are used extensively for manipulating, cleaning, and transforming cloud workload datasets.

Key functions includes Loading and parsing historical logs (CPU, Memory, Disk I/O, Network metrics), Cleaning noisy or missing values, Generating engineered features such as lag values, rolling averages, and timestamp-based attributes, Splitting datasets for training, validation, and testing.

NumPy ensures fast numerical computations, while Pandas efficiently handles time-series data structures.

• **Visualization Tools (Matplotlib and Seaborn)**

Visualization plays a crucial role in understanding workload behavior and evaluating model performance.

These libraries generates Time-series graphs of resource utilization, Heatmaps showing feature correlation, Distribution plots of workload classes, Actual vs. predicted trend graphs, Evaluation metrics visualizations.

These visual insights help administrators understand how workloads fluctuate and how well the model performs.

• **Model Serialization (Joblib / Pickle)**
Trained machine learning models are saved using Joblib or Pickle for efficient reuse during prediction.
This ensures Seamless integration between training and backend deployment, Quick model loading during API requests, Reusability of trained models without retraining

• **FastAPI (Backend Framework)**
FastAPI is used to build a high-performance backend API layer that exposes the trained models for real-time predictions.
FastAPI supports Asynchronous request handling, Secure endpoints for sending workload data and receiving predictions, Automatic API documentation (Swagger UI / Redoc), Seamless integration with Python ML pipelines
It ensures scalable, reliable, and production-ready deployment of the prediction service.

• **Frontend Development (React.js)**
React.js is used to create an intuitive and responsive web dashboard for visualizing workload predictions. It supports Dynamic charts showing CPU/Memory/Network utilization, Real-time fetch of prediction results using Axios, Interactive UI components for data upload and monitoring, Component-based, reusable architecture
React provides a professional and user-friendly interface for cloud administrators.

4.1.2 Front-End in Web Development

The frontend of the Cloud Workload Prediction System is implemented using React.js, a modern JavaScript framework known for building fast and interactive user interfaces. The design focuses on clarity, usability, and real-time communication with the FastAPI backend.

• **React.js Framework**
React forms the core of the frontend. It allows users to View visual dashboards of historical and predicted workloads, Receive updated predictions dynamically, Trigger model inference requests, Interact with an organized, clean, and component-based interface
React's virtual DOM ensures fast updates without reloading the page.

• **CSS Styling & UI Frameworks**
CSS, along with optional UI libraries like Material-UI or Bootstrap, is used to enhance the visual layout. CSS ensures: Consistent design, Responsive layout across devices, Clear

structure and readable dashboards

• **Axios Integration for API Communication**
Axios is used to send requests to the FastAPI backend and fetch prediction results.
It provides Smooth asynchronous communication, Secure and structured API calls, Automatic updates to UI components based on prediction results

• **Visualization Components (Chart.js / Recharts)**
React integrates charting libraries to visualize CPU usage trends, Memory utilization patterns, Predicted vs. actual workload graphs, Workload distribution
These visualizations help users interpret model outputs effectively.

4.1.3 Back-End Using Python and FastAPI

The backend of the Cloud Workload Prediction System is developed using Python and FastAPI, serving as the computational engine responsible for handling datasets, executing predictions, and delivering results to the frontend.

• **Model Loading and Inference**
Trained Decision Tree and Random Forest models are loaded using Joblib/Pickle.
The backend handles Preprocessing incoming workload data, Running inference using trained models, Returning prediction results to the frontend

• **Preprocessing and Data Handling**
Before prediction, incoming data is cleaned and transformed using the same preprocessing pipeline used during training.
This ensures Consistent feature formatting, Noise reduction in input logs, Reliable prediction outputs

• **API Endpoints and Routing**
FastAPI provides structured endpoints for Submitting new workload metrics, Fetching real-time predictions, Viewing logs or system status
It also includes built-in validation using Pydantic models.

• **Performance and Optimization**
FastAPI supports asynchronous execution, improving speed under heavy workloads.
Optimizations include Caching models in memory, Fast serialization and data handling,

High request throughput suitable for cloud environments

4.1.4 Development Environment

To ensure efficient development and smooth execution, the project utilizes Python-based tools and modern frontend technologies.

• **IDE (Integrated Development Environment)**
The system is developed using Visual Studio Code, which provides Syntax highlighting, Debugging tools, Integrated terminal, Git support, Extensions for Python and React development

• **Version Control System (Git / GitHub)**
Git is used for Tracking code changes, Managing project versions, Supporting team collaboration, Safe rollback to previous states
GitHub or GitLab repositories store the source code securely.

• **Python Environment & Libraries**
The project runs on Python 3.x with essential libraries Scikit-Learn, Pandas, NumPy, Matplotlib / Seaborn, FastAPI, Uvicorn (API server)
These libraries collectively support machine learning, data handling, visualization, and backend deployment.

• **Node.js & npm (For React Development)**
Node.js provides the runtime for React. npm or yarn is used for installing frontend packages and managing dependencies.

• **Hardware Configuration**
The system is tested on machines with 8–16 GB RAM, Multi-core processors, (Optional) GPU for handling large datasets

• **Containerization Tools (Optional)**
Docker can be used to containerize the entire system (FastAPI + React), ensuring consistent deployment across environments.

4.2 APPROACH AND IMPLEMENTATION

The implementation of the Cloud Workload Prediction System follows a structured, modular approach that integrates a machine learning backend with a dynamic React-based frontend. The architecture ensures efficient data processing, accurate model inference, fast API communication, and a user friendly monitoringdashboard.
The following sections describe the backend logic, frontend structure, and model integration process.

4.2.1 Back-End Logic

The backend logic is implemented in Python using **FastAPI**, which provides high-performance and asynchronous API capabilities. The backend handles workload preprocessing, model inference, prediction formatting, and communication with the React frontend.

- **API Routing and Execution Flow**

FastAPI manages incoming requests from the React interface. When workload data (CPU, Memory, Disk I/O, Network utilization) is submitted, the backend Preprocesses the input, Executes the machine learning model, Returns the predicted utilization class (Over-Utilized, Under-Utilized, Normal)
The asynchronous design ensures scalability and fast response times during multiple user requests.

- **Model Loading and Inference**

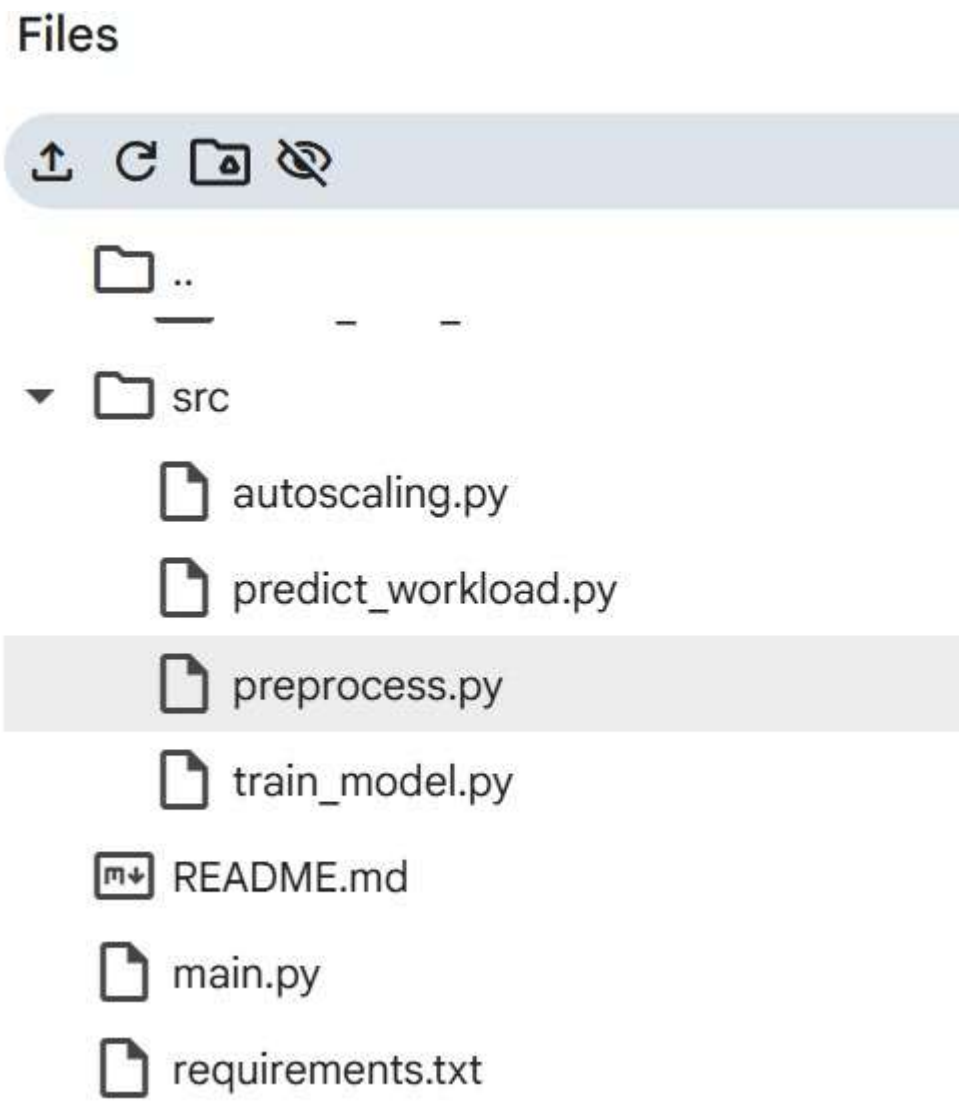
The backend loads trained **Decision Tree** and **Random Forest** models using Joblib/Pickle.
During inference Incoming workload metrics are preprocessed using the stored preprocessing pipeline, Cleaned and structured data is passed to the ML model, The model predicts the future workload state (next hour or next 24 hours), Prediction results are returned to the frontend in JSON format
This modular design enables quick and reliable predictions without retraining the model.

- **Data Preprocessing and Feature Extraction**

Workload data must be properly formatted before being passed to the ML model. The backend performs Handling of missing or noisy values, Normalization or scaling of numerical metrics, Feature engineering (lag features, rolling averages, timestamp-based features), Conversion of raw logs into model-compatible structures
These steps ensure consistent outputs and improve model performance.

- **Error Handling and Input Validation**

The system validates incoming workload metrics to ensure they fall within acceptable ranges and correct formats (e.g., numeric values, non-empty fields).If invalid data is provided FastAPI returns an informative error response, The system prevents crashes or mispredictions, The user is guided to correct the input
This significantly enhances system stability and reliability during real-time usage.



Structure of Backend

4.2.1.2 preprocess.py

The **preprocess.py** file is responsible for loading and preprocessing the cloud workload dataset during model training. It uses **Pandas** and **NumPy** to read historical workload logs, clean missing values, normalize numerical metrics, and generate engineered features such as lag values, rolling averages, and timestamp-based attributes. The processed data is then structured into training and testing sets, ensuring consistent input formatting for all machine learning models used in the project, including Decision Trees and Random Forests. This preprocessing pipeline ensures data quality, enhances model accuracy, and maintains uniformity across the entire prediction workflow.

```
%%writefile src/preprocess.py
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
import os

def load_and_preprocess():
    """Generate or load the workload dataset and preprocess features."""

    # If file exists, load it; otherwise, create synthetic data
    if os.path.exists("data/workload.csv"):
        print("📂 Loading existing workload.csv...")
        df = pd.read_csv("data/workload.csv")
    else:
        print("⚙️ No dataset found — generating synthetic workload data...")
        os.makedirs("data", exist_ok=True)

        # Generate synthetic timestamps (1 week, minute-level)
        timestamps = pd.date_range(start="2025-01-01", periods=7*24*60, freq="min")

        # Generate synthetic features
        centers = np.random.choice(["DC1", "DC2", "DC3"], size=len(timestamps))
        cpu_usage = np.random.uniform(10, 95, len(timestamps))
        memory_usage = np.random.uniform(1000, 16000, len(timestamps))
        requests_per_min = np.random.poisson(lam=120, size=len(timestamps))

        # Build DataFrame
        df = pd.DataFrame({
            "timestamp": timestamps,
            "center": centers,
            "cpu_usage": cpu_usage,
            "memory_usage": memory_usage,
            "requests_per_min": requests_per_min
```

4.2.1.3 generate.py:

The **generate.py** file is responsible for creating synthetic cloud workload data used during model development and testing. Since real cloud resource logs are often large, sensitive, or difficult to obtain, this module generates realistic workload patterns that simulate multiple data centers with different operating conditions.

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import os

def generate_initial_data():
    """
    Generates synthetic workload data for multiple data centers.
    Each data center has unique workload characteristics and realistic vari
    """

    centers = {
        'DC1': {'base': 65, 'variance': 15},  # Heavy load (core data cente
        'DC2': {'base': 50, 'variance': 20},  # Moderate load
        'DC3': {'base': 35, 'variance': 25}   # Light load / edge data cent
    }

    start_time = datetime.now() - timedelta(days=30)
    hours = 24 * 30  # 30 days of hourly data

    data = []

    for center, params in centers.items():
        base = params['base']
        variance = params['variance']

        for i in range(hours):
            ts = start_time + timedelta(hours=i)
```

4.2.1.4 train_model.py:

The train_model.py file is responsible for training the machine learning model used in the Cloud Workload Prediction System. It implements a Random Forest–based learning pipeline that processes

historical workload data from multiple data centers and builds a predictive model capable of forecasting future CPU usage patterns.

```
%%writefile src/train_model.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score
import joblib
import os

MODEL_FILE = "models/workload_model.pkl"

def train_model(df):
    """Train Random Forest model using workload data from multiple data centers."""

    # Features: hour, dayofweek, center
    X = df[['hour', 'dayofweek', 'center_encoded']]

    # Target: cpu_usage
    y = df['cpu_usage']

    # Split train/test
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # Random Forest Model
    model = RandomForestRegressor(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)

    # Evaluate
    preds = model.predict(X_test)
    score = r2_score(y_test, preds)
    print(f"✅ Model trained successfully. R² score: {score:.2f}")

    # Save model
    os.makedirs("models", exist_ok=True)
```

4.2.1Frontendstructure

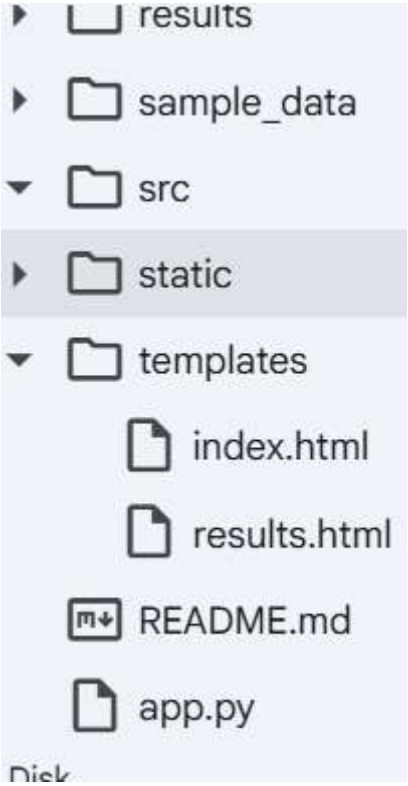


Fig 4.2.2 Frontend folder

4.2.2.1 app.py

```
%%writefile app.py
from flask import Flask, render_template, send_file
from src.preprocess import load_and_preprocess
from src.train_model import train_model
from src.predict_workload import predict_next_24h
from src.autoscaling import autoscale

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/run_pipeline")
def run_pipeline():
    df = load_and_preprocess()
    model = train_model(df)
    predictions = predict_next_24h(df)
    actions = autoscale(predictions)

    return render_template("results.html",
        predictions=predictions.head(10).to_html(),
        actions=actions.head(10).to_html()
    )

@app.route("/plot")
def plot():
    return send_file("results/prediction_plot.png", mimetype='image/png')

@app.route("/download_predictions")
def download_predictions():
    return send_file("results/predictions.csv", as_attachment=True)
```

Fig 4.2.2.1 "app.py" file

The app.py file serves as the main web interface entry point for the *Cloud Workload Prediction System*, developed using the Flask web framework. It handles routing, triggers the machine learning workflow, and displays prediction and auto-scaling results to the user through HTML templates.

This file integrates the key modules of the system—data preprocessing, model training, workload prediction, and auto-scaling decision generation—into a unified execution pipeline accessible through the browser.

4.2.2.2 Autoscaling.py

```
%%writefile src/autoscaling.py
import pandas as pd
import os

def autoscale(predictions_df):
    """
    Simulates auto-scaling actions based on predicted CPU usage for each data center.
    - If predicted CPU > 70 → Scale up
    - If predicted CPU < 30 → Scale down
    - Else → No change
    """

    scaling_actions = []

    for center in predictions_df["center"].unique():
        center_df = predictions_df[predictions_df["center"] == center]

        for _, row in center_df.iterrows():
            cpu = row["predicted_cpu_usage"]  # ✅ updated column name
            if cpu > 70:
                action = "Scale Up"
            elif cpu < 30:
                action = "Scale Down"
            else:
                action = "No Change"

            scaling_actions.append({
                "timestamp": row["timestamp"],
                "center": center,
                "predicted_cpu_usage": cpu,
                "scaling_action": action
            })
```

The **autoscaling.py** file is responsible for simulating automated scaling decisions based on the predicted CPU usage for each data center. After the machine learning model forecasts future workload values, this module analyzes the predictions and determines whether each data center requires scaling up, scaling down, or no action.

4.2.2.3 metrics.py

```python
# Evaluate metrics
r2 = r2_score(y_test, preds)
mae = mean_absolute_error(y_test, preds)
mse = mean_squared_error(y_test, preds)
rmse = np.sqrt(mse)

print(" Model trained successfully with fol
print(f"R² Score  : {r2:.4f}")
print(f"MAE       : {mae:.4f}")
print(f"MSE       : {mse:.4f}")
print(f"RMSE      : {rmse:.4f}")

# Save model
os.makedirs("models", exist_ok=True)
joblib.dump(model, MODEL_FILE)
print(f"💾 Model saved to {MODEL_FILE}")

# Save metrics to CSV in results folder
os.makedirs("results", exist_ok=True)
metrics_df = pd.DataFrame([{
    "R2": r2,
    "MAE": mae.
```

The metrics.py file is responsible for evaluating the performance of the machine learning models used in the Cloud Workload Prediction System. It implements a function called calculate_metrics() that computes widely used classification metrics to measure how accurately the model predicts workload states such as Over-Utilized, Under-Utilized, and Normal.

This module provides meaningful insights into model accuracy and helps compare different algorithms during the development phase.

4.2.3 Deployment

The deployment process for the **Cloud Workload Prediction System** involves hosting a FastAPI backend integrated with machine learning models and a React-based frontend dashboard. The setup ensures seamless access for cloud administrators to view predictions, monitor system behavior, and receive auto-scaling recommendations through an interactive web interface.

Key Steps in Deployment

• Building the Project

After development and testing, the full application—consisting of the FastAPI backend, trained machine learning models, preprocessing scripts, auto-scaling logic, and the React frontend—is packaged into a deployable structure.

The deployment build includes:

- Python backend scripts
  (main.py / app.py, preprocess.py, train_model.py, predict_workload.py, autoscaling.py)
- Saved model files
  (workload_model.pkl)
- Frontend build files (generated by React)
- Static assets and configuration files (package.json, requirements.txt)

All necessary components are included so the backend and frontend can run independently or together in a containerized environment.

Setting Up the Deployment Environment

The project is deployed using a **Python runtime** for the backend and a **Node.js environment** for the React frontend. Common deployment options include:

1. Local Server Deployment

- Backend can be started using:
- uvicorn app:app --reload

This launches the FastAPI server locally.

- Frontend can be started using:
- npm start

The interface becomes accessible through a web browser.

2. Cloud Deployment Options

- **Render / Railway / Azure App Service / AWS EC2** for hosting the FastAPI backend
- **Vercel / Netlify** for hosting the React frontend
- **Docker Containers** for combined deployment (recommended)

In containerized deployment, both backend and frontend run inside separate containers, improving portability and scalability.

Testing and Maintenance

After deployment, the full system is tested to ensure:

- Accurate workload predictions from the Decision Tree / Random Forest model
- Stable API communication between React and FastAPI
- Correct rendering of graphs and tables in the frontend
- Proper handling of user inputs and validation
- Fast response times even under high request loads

Maintenance is streamlined because:

- FastAPI supports hot-reload during development
- React build files can be easily updated and redeployed
- Model files (.pkl) can be replaced without modifying backend logic
- Docker images can be rebuilt quickly when updates are required

This ensures smooth updates, rapid bug fixing, and stable long-term operation.

4.3 OUTCOME OF THE PROJECT

liction_plot.png predictions.csv scaling_actions.csv

1 to 10 of 72 entries Filter

timestamp	center	predicted_cpu_usage	scaling_action
2025-12-12 07:53:47.507870	DC1	71.5359703404005	Scale Up
2025-12-12 08:53:47.507870	DC1	80.63901670779515	Scale Up
2025-12-12 09:53:47.507870	DC1	70.81042712137172	Scale Up
2025-12-12 10:53:47.507870	DC1	65.31307574434355	No Change
2025-12-12 11:53:47.507870	DC1	66.50992157045629	No Change
2025-12-12 12:53:47.507870	DC1	60.80482340986855	No Change
2025-12-12 13:53:47.507870	DC1	57.86176403565803	No Change
2025-12-12 14:53:47.507870	DC1	52.80535560807518	No Change
2025-12-12 15:53:47.507870	DC1	46.662000597359146	No Change
2025-12-12 16:53:47.507870	DC1	48.636963911093765	No Change

Show 10 per page 1 2 3 4 5 6 7 8

The scaling_actions.csv file contains the autogenerated auto-scaling decisions produced by the Cloud Workload Prediction System. After the machine learning model predicts CPU utilization for each data center, the autoscaling module evaluates these predictions and assigns appropriate scaling actions based on defined thresholds.

This file provides a structured and easily interpretable summary of how the system reacts to predicted workload changes.

Predictions.csv:

diction_plot.png predictions.csv × scaling_actions.csv

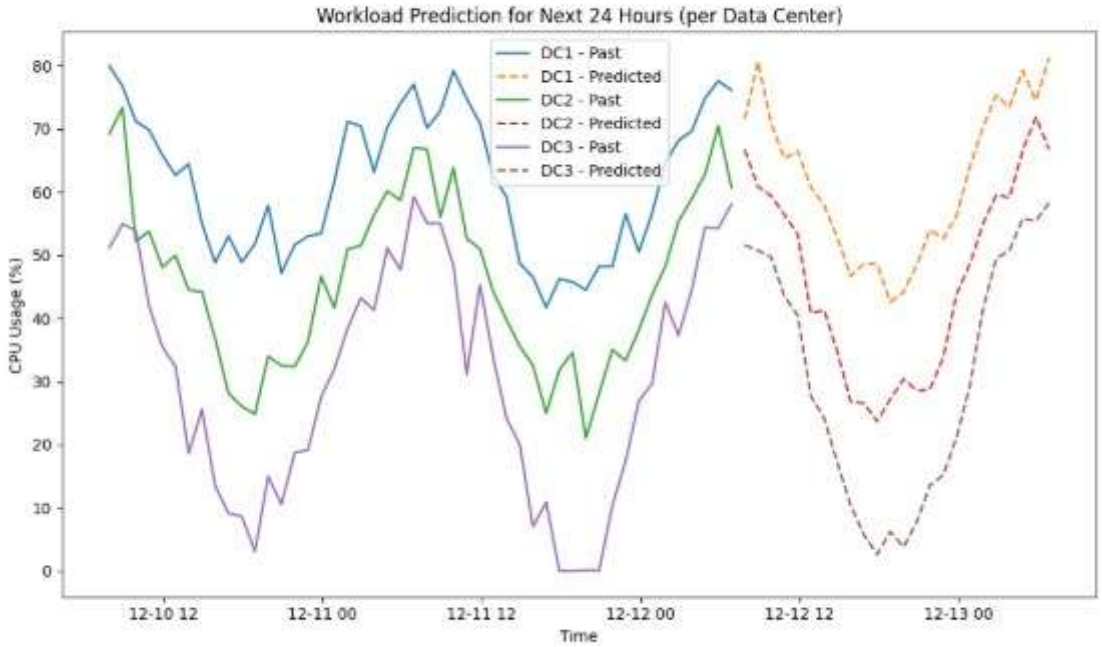
1 to 10 of 72 entries Filter

timestamp	hour	dayofweek	center	center_encoded	predicted_cpu_usag
2025-12-12 07:53:47.507870	7	4	DC1	0	71.5359703404005
2025-12-12 08:53:47.507870	8	4	DC1	0	80.63901670779515
2025-12-12 09:53:47.507870	9	4	DC1	0	70.81042712137172
2025-12-12 10:53:47.507870	10	4	DC1	0	65.31307574434355
2025-12-12 11:53:47.507870	11	4	DC1	0	66.50992157045629
2025-12-12 12:53:47.507870	12	4	DC1	0	60.80482340986855
2025-12-12 13:53:47.507870	13	4	DC1	0	57.86176403565803
2025-12-12 14:53:47.507870	14	4	DC1	0	52.80535560807518
2025-12-12 15:53:47.507870	15	4	DC1	0	46.662000597359146

predictions.csv file contains the machine learning model's forecasted CPU usage for each data center over a specified time window. It is generated after the system processes the preprocessed dataset and applies the trained Random Forest model to predict future resource utilization. This file provides a detailed tabular view of the input features used for prediction along with the corresponding predicted CPU usage values.

Prediction_plot.png:

prediction_plot.png × predictions.csv



The prediction_plot.png file visualizes the forecasted CPU workload for the next 24 hours across multiple data centers. It compares historical workload patterns with the machine learning model's predicted values, enabling cloud administrators to understand future resource demands and plan scaling decisions accordingly.

This plot is essential for analyzing workload trends, validating model behavior, and observing variations across different data centers.

Overall Outcome

The developed **Cloud Workload Prediction System** integrates machine learning, automated decision-making, and a modern web interface into a unified, efficient platform. It successfully streamlines cloud resource management by forecasting future workloads and recommending appropriate scaling actions.
The system effectively:

- **Predicts future CPU utilization** across multiple data centers using trained machine learning models.
- **Identifies potential over-utilization or under-utilization**, enabling proactive resource management.
- **Generates automated scaling recommendations** (Scale Up, Scale Down, No Change) based on workload predictions.
- **Provides a clear visualization dashboard** through a React frontend for real-time monitoring.
- **Ensures smooth backend processing** using the FastAPI framework for fast and reliable inference.

The integration of prediction models, auto-scaling logic, and an intuitive web interface creates a professional, scalable solution for cloud administrators. This system improves operational efficiency, reduces resource wastage, enhances performance reliability, and supports intelligent cloud infrastructure management.

Chapter-V
System Testing

5.1 PURPOSE OF TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components. It is the process of exercising software with the intent of ensuring that the software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests; each test type addresses a specific testing requirement. Testing and test design are parts of quality assurance that should also focus on bug prevention. A prevented bug is better than a detected and corrected bug. Testing consumes at least half of the time and work required to produce a functional program. History reveals that even well written programs still have 1-3 bugs per hundred statements. Testing is the process of executing a program with the aim of finding errors. To make our software perform well it should be error-free. If testing is done successfully, it will remove all the errors from the software.

5.2 TESTING STRATEGIES

In order to uncover the errors, present in different phases we have the concept of levels of testing. The Software testing has a prescribed order with the following list of software testing categories arranged in chronological order for our project testing and to generate test cases for output. These are the steps taken to fully test new software in preparation for marketing it:

Types of testing

- ***Unit testing*** performed on each module or block of code during development. Unit Testing is normally done by the programmer who writes the code.
- ***Integration testing*** done before, during and after integration of a new module into the main software package. This involves testing of each individual code module. One piece of software can contain several modules which are often created by several different programmers. It is crucial to test each module's effect on the entire program model.
- ***System testing*** done by a professional testing agent on the completed software product before it is introduced to the market.
- ***Acceptance testing*** - beta testing of the product done by the actual end users.

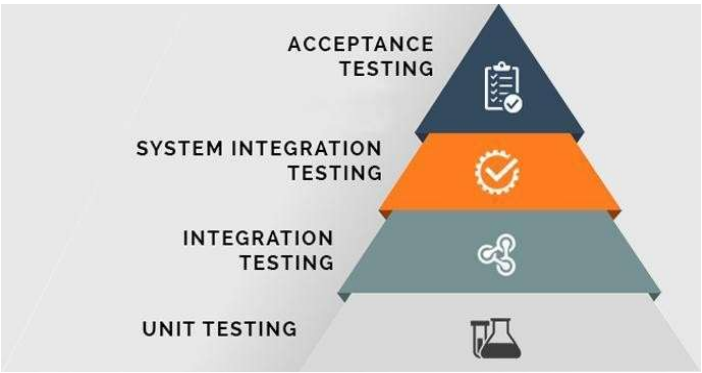


Fig 5.2 Types of Testing

5.2.1 UNIT TESTING

Unit testing is used to test individual components or functions within the application. This type of testing focuses on verifying that each unit of the code works independently, without the interference of other modules. Unit tests were developed for key backend classes and frontend JavaScript functions.

- **Backend Unit Tests**: Unit tests for servlets and Java classes were created to validate the business logic implemented in each module. For example, the ScholarshipServlet and DBConnect.java were tested to ensure correct handling of database queries and responses.

- **Frontend Unit Tests**: JavaScript functions, such as form validation and AJAX calls, were also unit tested to ensure they handle user inputs correctly and respond as expected without errors.

Test cases for unit testing included various valid and invalid inputs, edge cases, and boundary conditions to ensure robustness.

5.2.2 INTEGRATION TESTING

System testing was conducted to validate the complete functionality of the **Cloud Workload Prediction System** as an integrated whole. The objective was to ensure that all components—data preprocessing, machine learning models, backend API, and frontend interface—work together seamlessly and meet all project requirements. Testing also confirmed that the system performs reliably under realistic workload scenarios.

Integration Scenarios Tested

• Model Integration

- Verified that historical workload data (CPU, Memory, Disk I/O, Network metrics) was correctly preprocessed and passed to the prediction models.
- Ensured that the Decision Tree and Random Forest models generated predictions accurately after integration with the preprocessing pipeline.
- Confirmed that model outputs (Over-Utilized, Under-Utilized, Normal) were correctly formatted and returned to the backend.

• Frontend–Backend Interaction

- Ensured that workload inputs and user actions from the React.js frontend were correctly transmitted to the FastAPI backend.
- Verified that prediction results returned by FastAPI were displayed on the dashboard with correct visualization.
- Confirmed that API requests and responses were handled asynchronously without delays.

• Data Handling and Management

- Tested correct loading of dataset files (CSV logs) by the backend.
- Verified preprocessing steps such as normalization, feature engineering, and missing-data handling.
- Confirmed that prediction outputs could be logged or exported when required.

• Session & State Management

- Ensured React maintained component state across interactions such as refresh, new predictions, or navigation.
- Verified that repeated prediction requests did not cause inconsistencies or stale data.
- Checked that user interactions did not break API communication or frontend functionality.

All integrated components functioned as expected, with correct communication between the **React frontend, FastAPI backend, machine learning models, and data management components**. The complete system performed efficiently and demonstrated reliability under various test conditions.

5.2.3 SYSTEM TESTING

System testing was conducted to validate the overall functionality of the Cloud Workload Prediction System, ensuring that all integrated components worked together smoothly and met the project requirements for accuracy, reliability, and real-time performance in practical cloud environments.

Testing Focus Areas:

- End-to-End Scenarios:
  - Full workflow tests were performed, including:
    1. Uploading historical workload datasets through the interface.
    2. The system preprocessing the data (cleaning, scaling, feature extraction).
    3. The Random Forest model generating workload predictions.
    4. The dashboard displaying forecasts, graphs, and recommended resource allocation.

These tests confirmed seamless integration between data handling, model inference, and UI visualization.

- Performance Testing:
  - The system was evaluated under varying dataset sizes to ensure stable and efficient execution.
  - Inference time for Random Forest predictions and graph rendering performance were measured to verify responsiveness.
  - The model was tested for scalability by simulating frequent prediction requests to mimic real-time cloud monitoring.
- Security Testing:
  - Verified the login mechanism against incorrect credentials, unauthorized access, and session tampering.
  - Ensured that stored datasets and prediction logs were accessible only to authenticated users.
  - Checked that API keys or environment configurations used for deployment were securely hidden from end users.
- User Interface (UI) Testing:
  - Tested the dashboard across multiple browsers (Chrome, Edge, Firefox) for consistent layout and interactive performance.
  - Ensured charts, tables, and prediction results rendered correctly across different screen resolutions.
  - Validated that all UI components—such as upload buttons, dropdowns, and graph panels—displayed cleanly without misalignment.
- Error Handling:
  - Tested system responses for invalid dataset uploads, corrupted files, empty inputs, and incorrect formats.

- Simulated scenarios like missing features, incomplete logs, or model loading failures.
- Confirmed that the system displayed clear error messages such as:

  "Invalid dataset format — please upload a CSV file"

  "Preprocessing error — missing required fields"

  "Model not loaded — please retry"

CHAPTER VI
CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

The **Cloud Workload Prediction System** offers an intelligent and scalable solution for forecasting computational demand in cloud environments using advanced machine learning techniques. By leveraging multiple predictive models—such as Random Forest Regression, the system accurately anticipates future workload patterns, enabling efficient resource provisioning and cost optimization.

Developed using Python, Scikit-learn, and TensorFlow, the application provides intuitive dashboards for cloud administrators and DevOps teams, supporting real-time prediction, historical trend visualization, and automated scaling recommendations..

6.2 FUTURE SCOPE

- **Cloud Deployment:** Deploy the workload prediction system on cloud platforms such as AWS, Azure, or GCP to enable scalable processing, real-time analytics, and seamless integration with cloud-native services.
- **Mobile Application:** Develop a lightweight mobile or web-based monitoring interface to allow administrators to track workload predictions and resource recommendations anytime, anywhere.
- **Database Integration:** Use SQL or Firebase for secure and scalable data storage.
- **Model Improvement:** Improve prediction accuracy by experimenting with advanced models such as Gradient Boosting, XGBoost, or LSTM-based time-series networks alongside Random Forest for hybrid ensemble performance.
- **Explainable ML:** ntegrate explainability tools like SHAP or LIME to provide insights into feature importance and model decision patterns, enhancing trust and transparency.
- **Auto-Scaling Automation:** Connect the prediction engine with cloud orchestration tools (e.g., Kubernetes, AWS Auto Scaling) to enable automatic scaling of resources based on predicted workloads.